

UNIVERSIDAD CONTINENTAL

FACULTAD DE INGENIERÍA

ESCUELA ACADÉMICO PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA



PROYECTO

**“Plataforma Web de Capacitación, E-commerce y Gestión de
Calidad para la Industria Láctea”**

PRESENTADO POR:

APELLIDOS Y NOMBRES	CÓDIGO
Omar Piero Terbullino Jaime	72682019
Giancarlo Guerrero Córdova	71993692
Canchumanya Avellaneda Roger Moises	71653804

ASESOR:

DR. MAGLIONI ARANA CAPARACHIN

HUANCAYO – PERÚ

2026

CONTENIDO GENERAL

PORTADA	i
CONTENIDO GENERAL	ii
CAPÍTULO 1	v
INFORMACIÓN GENERAL DEL PROYECTO	v
1.1. Resumen Ejecutivo	v
1.2. Introducción	vi
CAPÍTULO 2	vi
CONTEXTO ORGANIZACIONAL Y ANÁLISIS DEL PROBLEMA	vi
2.1. Contexto de la Organización	vi
2.2. Identificación del Problema	vi
CAPÍTULO 3	vii
ANÁLISIS DE PROCESOS DE NEGOCIO	vii
3.1. Descripción del Proceso Actual	vii
3.2. Modelado del Proceso Actual (AS-IS)	vii
3.3. Problemas del Proceso Actual	vii
3.4. Modelado del Proceso Propuesto (TO-BE)	viii
CAPÍTULO 4	viii
ANÁLISIS DE REQUERIMIENTOS DEL SISTEMA	viii
4.1. Identificación de Actores del Sistema	viii
4.2. Requerimientos Funcionales	viii
4.3. Requerimientos no Funcionales	ix
4.4. Casos de Uso del Sistema	ix
CAPÍTULO 5	ix
PLANIFICACIÓN DEL PROYECTO Y PLAN DE CALIDAD	ix
5.1. Alcance del Proyecto	ix
5.2. Herramientas Tecnológicas del Proyecto	ix

5.3. Normas y Estándares de Calidad	x
5.4. Plan de Pruebas del Proyecto	x
5.5. Lineamientos de Seguridad Informática	x
CAPÍTULO 6	x
DISEÑO DEL SISTEMA	x
6.1. Arquitectura Conceptual del Sistema	xi
6.2. Modelo UML del Sistema	xi
6.3. Diseño de Interfaces de Usuario	xi
6.4. Diseño de Base de Datos	xi
CAPÍTULO 7	xii
ARQUITECTURA TECNOLÓGICA DEL SISTEMA	xii
7.1. Tecnologías del Frontend	xii
7.2. Tecnologías del Backend	xii
7.3. Base de Datos del Sistema	xii
7.4. Infraestructura de Desarrollo	xiii
CAPÍTULO 8	xiii
DESARROLLO DEL SISTEMA	xiii
8.1. Iteración 1: Configuración Inicial del Proyecto	xiii
8.2. Iteración 2: Desarrollo de Funcionalidades Básicas	xiii
8.3. Iteración 3: Implementación de Módulos Funcionales	xiii
8.4. Iteraciones Posteriores	xiv
CAPÍTULO 9	xiv
CONTROL DE VERSIONES Y GESTIÓN DEL REPOSITORIO	xiv
9.1. Repositorio del Proyecto	xiv
9.2. Estrategia de Control de Versiones	xiv
9.3. Gestión de Ramas del Proyecto	xiv
9.4. Registro de Commits Relevantes	xv
CAPÍTULO 10	xv

DOCKERIZACIÓN Y DESPLIEGUE DE MÓDULOS	xv
10.1. Introducción a Docker en el Proyecto	xv
10.2. Dockerización del Backend	xv
10.3. Dockerización del Frontend	xv
10.4. Orquestación con Docker Compose	xv
CAPÍTULO 11	xvi
ESTRATEGIA DE PRUEBAS DE SOFTWARE	xvi
11.1. Enfoque de Pruebas del Proyecto	xvi
11.2. Niveles de Pruebas Aplicados	xvi
11.3. Tipos de Pruebas Ejecutadas	xvi
11.4. Plan de Ejecución de Pruebas	xvii
CAPÍTULO 12	xvii
AUTOMATIZACIÓN DE PRUEBAS	xvii
12.1. Herramientas de Automatización	xvii
12.2. Configuración del Entorno de Pruebas	xvii
12.3. Scripts de Pruebas Automatizadas	xvii
12.4. Ejecución Automática de Pruebas	xviii
CAPÍTULO 13	xviii
MÉTRICAS DE CALIDAD	xviii
13.1. Ejecución de Casos de Pruebas	xviii
13.2. Registro de Defectos	xviii
13.3. Métricas de Calidad del Software	xviii
13.4. Evaluación de Calidad basada en Estándares	xix
CAPÍTULO 14	xix
IMPLEMENTACIÓN Y MONITOREO	xix
14.1. Preparación del Entorno de Implementación	xix
14.2. Implementación del Sistema	xix
14.3. Verificación de Funcionamiento	xix

14.4. Monitoreo del Sistema	xx
CONCLUSIONES	xx
RECOMENDACIONES	xx
REFERENCIAS	xx
ANEXOS	xxi

CAPÍTULO 1

INFORMACIÓN GENERAL DEL PROYECTO

1.1. Resumen Ejecutivo

El presente proyecto consiste en el desarrollo de una plataforma web para la empresa **JM y JS Alimentos Lácteos**, con el propósito de fortalecer su presencia digital, mejorar la captación y fidelización de clientes, y difundir sus servicios de asesoría en gestión de calidad, buenas prácticas de manufactura y capacitaciones dirigidas al sector alimentario. La problemática identificada se relaciona con la limitada visibilidad de la empresa en medios digitales, debido a la escasa actualización de redes sociales, la falta de contenido dinámico y la ausencia de personal especializado en marketing digital, lo que restringe el alcance de sus productos y servicios.

Como solución tecnológica, se propone una página web informativa e interactiva que permita presentar los servicios de la empresa, compartir contenidos como documentos y videos, y facilitar la comunicación directa con los usuarios mediante WhatsApp. La plataforma está orientada a ofrecer una navegación sencilla, didáctica y accesible para pequeñas y medianas empresas, profesionales y estudiantes vinculados al rubro alimentario.

En cuanto a las tecnologías, el proyecto se apoya en herramientas de desarrollo web orientadas a construir un sistema funcional, seguro y escalable. Desde el enfoque de calidad de software, se priorizan atributos como la **seguridad**, la **fluidez del sistema** y la **confiabilidad** en los procesos críticos, especialmente en las transacciones relacionadas con pagos, las cuales deben ejecutarse sin errores ni retrasos.

Como resultado, se espera que la plataforma contribuya al posicionamiento digital de la empresa, optimice la interacción con los clientes y fortalezca su proyección como organización líder en el rubro de productos lácteos y asesoría en calidad alimentaria.

1.2. Introducción

En la actualidad, la ingeniería de software permite desarrollar soluciones tecnológicas que optimizan procesos y fortalecen la relación entre las organizaciones y sus clientes. En este marco, el presente proyecto propone el desarrollo de una plataforma web para la empresa **JM y JS Alimentos Lácteos**, con el fin de mejorar su presencia digital, difundir sus servicios de asesoría y capacitaciones, y facilitar la interacción con los usuarios. La necesidad del sistema surge por las limitaciones actuales en la actualización de contenido y en el alcance de sus medios digitales.

Asimismo, las pruebas de software cumplen un rol esencial en el desarrollo moderno, ya que permiten identificar errores y asegurar atributos de calidad como la seguridad, la confiabilidad y la usabilidad. En este proyecto, su aplicación se orienta a garantizar el correcto funcionamiento del sistema, especialmente en procesos críticos como la interacción del cliente y las transacciones.

El propósito de la plataforma es ofrecer un entorno web informativo e interactivo que permita presentar productos, servicios, cursos y materiales multimedia, además de facilitar la comunicación con los clientes. El informe se organiza en secciones que describen el contexto del proyecto, la solución propuesta, el enfoque de calidad aplicado, las pruebas realizadas y los resultados obtenidos.

CAPÍTULO 2

CONTEXTO ORGANIZACIONAL Y ANÁLISIS DEL PROBLEMA

Este capítulo describe el entorno donde se identifica el problema y explica por qué es necesario desarrollar una solución informática.

2.1. Contexto de la Organización

El análisis del escenario organizacional y la implementación de la nueva plataforma se fundamentan en estándares internacionales de calidad para asegurar la viabilidad del modelo de negocio:

- **Alineación con ISO/IEC 25010** : La transición hacia una infraestructura propia busca garantizar atributos críticos como la Seguridad y la Eficiencia de Desempeño. En el escenario actual, la validación manual de pagos representa un riesgo de seguridad y posibles latencias (delays), lo cual vulnera la confianza del usuario. El nuevo sistema permitirá automatizar estas funciones para cumplir con los niveles de calidad técnica requeridos por la norma.
- **Enfoque en la Satisfacción del Cliente:** De acuerdo con los principios de la norma ISO 9000, el sistema debe asegurar una experiencia de usuario óptima. Actualmente, el dinamismo deficiente en redes sociales y la atención lenta por WhatsApp afectan la fidelización. La plataforma web se diseñará bajo criterios de Usabilidad para ofrecer una navegación intuitiva que garantice la satisfacción del cliente desde el primer contacto hasta la entrega del material educativo.
- **Mitigación de Riesgos Operativos:** El análisis QA (Aseguramiento de la Calidad) es vital para este escenario, ya que permite identificar fallos en el flujo de inscripción y asegurar que la habilitación del material educativo sea inmediata tras el pago, evitando los errores de operación interna que ocurren en el modelo manual actual

2.2. Identificación del Problema

El problema central radica en la **dependencia absoluta de procesos manuales** para la comercialización y entrega de servicios educativos en "JM y JS Alimentos Lácteos". Actualmente, la empresa carece de un ecosistema digital integrado, lo que obliga a que toda la interacción con el cliente (consultas, envío de información, validación de pagos y entrega de materiales) se realice de forma fragmentada a través de redes sociales estáticas y mensajería instantánea (WhatsApp). Esta falta de automatización impide que el negocio funcione de manera autónoma y profesional.

Causas del problema

Según el análisis realizado, las causas principales son:

- **Infraestructura tecnológica limitada:** Uso de herramientas no diseñadas para el e-commerce (Facebook y TikTok) que carecen de dinamismo y capacidad transaccional.
- **Centralización operativa:** El flujo completo de ventas recae sobre un único actor, el **Coordinador Digital**, quien debe intervenir manualmente en cada etapa.
- **Ausencia de validación automatizada:** Los pagos se verifican mediante la revisión manual de capturas de pantalla (vouchers), un método propenso a errores y demoras.
- **Falta de personal técnico especializado:** La inexistencia de un área de TI o Marketing digital que soporte una plataforma propia.

Consecuencias operativas

Esta situación genera efectos negativos directos en el día a día de la organización:

- **Cuellos de botella logísticos:** La saturación de mensajes en WhatsApp retrasa la respuesta al cliente, especialmente cuando aumenta el volumen de interesados.
- **Riesgo de errores humanos:** La inscripción de alumnos y la entrega de credenciales de acceso se realizan manualmente, lo que puede derivar en registros incorrectos o entrega de material equivocado.
- **Vulnerabilidad en la seguridad:** La gestión de transacciones mediante chats informales compromete la integridad de los datos financieros y la encriptación de la información de los clientes.

Impacto en la eficiencia del proceso

El impacto final se traduce en una **falta de escalabilidad**. Bajo el modelo actual (AS-IS), el esfuerzo requerido para concretar una sola venta es excesivamente alto, lo que hace imposible que la empresa pueda expandirse hacia el sector corporativo o grandes empresas, que es su visión a largo plazo. La "latencia" o *delay* en la habilitación del material educativo tras el pago degrada la percepción de calidad del producto peruano que la empresa busca promover, contraviniendo los principios de agilidad y eficiencia de la norma ISO 9000.

CAPÍTULO 3

ANÁLISIS DE PROCESOS DE NEGOCIO

Este capítulo analiza cómo funcionan los procesos organizacionales antes de implementar el sistema. Se utilizan técnicas de **Business Process Management (BPM)** para comprender el flujo de trabajo actual.

3.1. Descripción del Proceso Actual

Actores involucrados

- **Cliente / Profesional:** Estudiante o representante de una PyME interesado en adquirir conocimientos sobre parámetros de calidad láctea.
- **Redes Sociales (Facebook / TikTok):** Actúan como canales de difusión pasiva donde se exhibe el contenido.
- **Coordinador Digital:** Es el actor central de la organización, responsable de toda la gestión administrativa, comercial y operativa vía WhatsApp.

Actividades principales y Flujo de información

El proceso se ejecuta de manera lineal y manual a través de las siguientes etapas:

Difusión de Contenido: El flujo de información inicia en las **Redes Sociales**, donde la empresa publica anuncios o videos. El cliente recibe esta información de manera unidireccional.

Contacto Inicial: Al existir interés, el **Cliente** debe abandonar la red social y hacer clic en un enlace que lo redirige a **WhatsApp**. En este punto, la información es fragmentada y depende de la rapidez de respuesta del personal humano.

Gestión de Consultas: El **Coordinador Digital** recibe la solicitud y redacta manualmente la respuesta, enviando los detalles de los cursos, costos y métodos de pago.

Decisión de Compra: Se presenta una compuerta de decisión. Si el cliente desiste, el proceso finaliza. Si acepta, el flujo continúa con la solicitud de los datos bancarios de la empresa.

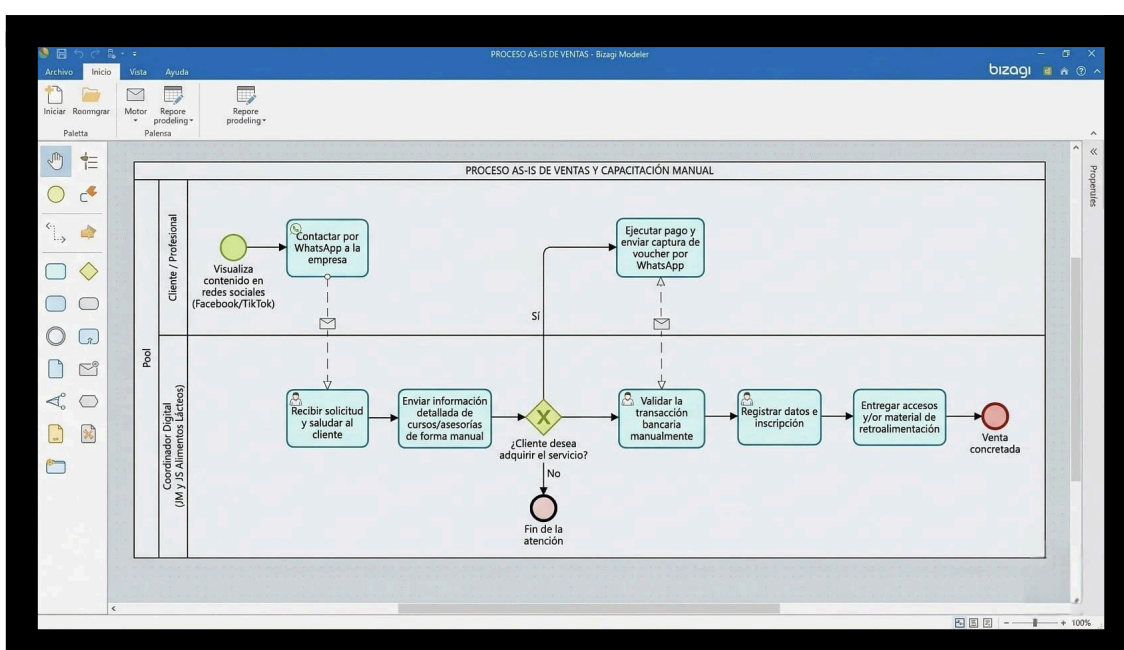
Ejecución y Envío de Pago: El cliente realiza la transferencia y, obligatoriamente, debe enviar una **captura de pantalla (voucher)** a través del chat de WhatsApp.

Validación Manual: El **Coordinador Digital** recibe el comprobante y debe verificar manualmente la transacción en la aplicación bancaria de la empresa para evitar fraudes o errores de depósito. Este es un punto crítico que genera retrasos (*delays*).

Registro de Inscripción: Tras confirmar el dinero, el coordinador solicita los datos personales al cliente y los anota manualmente en una base de datos local (hojas de cálculo o registros internos).

Entrega de Accesos: Finalmente, la información (enlaces a videos, documentos PDF o credenciales) fluye desde el coordinador hacia el cliente vía mensaje de texto, dando por finalizada la venta.

3.2. Modelado del Proceso Actual (AS-IS)



Nº	Actor	Tipo (Interno / Externo / Sistemas)	Lane asignado
1	Cliente / Profesional	Externo	Lane superior ("Cliente / Profesional")
2	Coordinador Digital	Interno	Lane inferior ("Coordinador Digital")

Puntos Críticos del Proceso Actual (AS-IS)

Estos son los momentos dentro del gráfico donde el proceso es más vulnerable a errores, retrasos o pérdida de clientes:

Dependencia del Canal Externo (WhatsApp):

Ubicación en el gráfico: Entre el contacto inicial y la recepción de información.

Riesgo: Al no ser un entorno controlado por la empresa, la información se fragmenta. Si el cliente no hace clic en el enlace de WhatsApp, el interés se pierde de inmediato sin dejar rastro en una base de datos.

Cuello de Botella en la "Atención Personalizada":

Ubicación en el gráfico: Tarea realizada por el **Coordinador Digital**.

Riesgo: Existe una centralización operativa crítica. Toda la capacidad de venta de la empresa depende de la velocidad de respuesta de una sola persona. Si hay múltiples solicitudes simultáneas, el tiempo de respuesta aumenta, lo que genera desinterés en el cliente.

Validación Manual de la Transacción (Punto Crítico de Control):

Ubicación en el gráfico: Actividad posterior a "Ejecutar pago".

Riesgo: Es el punto más sensible. El coordinador debe verificar manualmente el ingreso del dinero comparando capturas de pantalla con su banca móvil. Esto es propenso a errores humanos, fraudes con vouchers falsos y genera un **"delay" (retraso)** que afecta la percepción de calidad del servicio.

Habilitación de Accesos con Intervención Humana:

Ubicación en el gráfico: Tarea final antes de la "Venta concretada".

Riesgo: El cliente, tras haber pagado, debe esperar a que el coordinador esté disponible para enviarle los enlaces o materiales. Según la norma **ISO/IEC 25010**, esto afecta la "Eficiencia de desempeño" del servicio, ya que la entrega no es inmediata.

Ausencia de Registro Automatizado:

Ubicación en el gráfico: Actividad "Registrar datos e inscripción".

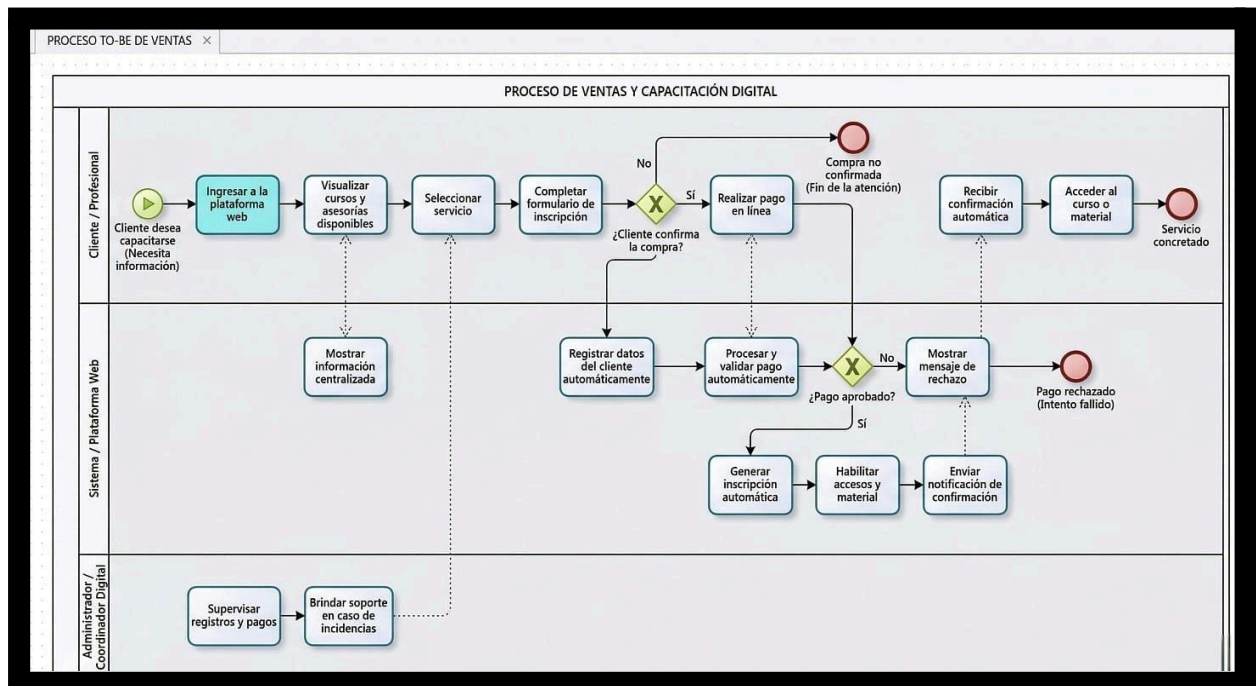
Riesgo: Al hacerse de forma manual, existe un alto riesgo de inconsistencia en los datos del cliente (errores en el nombre, correo o curso asignado), lo que dificulta cualquier labor de post-venta o soporte técnico posterior.

3.3. Problemas del Proceso Actual

A partir del análisis del proceso actual, se han detectado deficiencias críticas que limitan el crecimiento de la organización:

- **Procesos Manuales (Dependencia del Factor Humano):** La mayor debilidad radica en que el proceso es **"persona-dependiente"**. Desde la atención de consultas en WhatsApp hasta la validación de vouchers y el envío de materiales, cada paso requiere la intervención directa del Coordinador Digital. Esto impide que la empresa pueda procesar ventas en horarios no laborables o gestionar un alto volumen de clientes simultáneamente.
- **Duplicación de Tareas y Reprocesos:** Existe una redundancia ineficiente en la gestión de datos. El coordinador debe solicitar la información al cliente por chat, recibirla, y luego volver a digitarla manualmente en una base de datos local o registro de inscripciones. Esta transcripción manual no solo consume tiempo, sino que aumenta el riesgo de errores en los nombres o correos electrónicos, obligando a realizar correcciones posteriores (reprocesos).
- **Falta de Control de la Información:** Al utilizar herramientas externas como WhatsApp y redes sociales para la operación comercial, la información queda **fragmentada y descentralizada**. No existe un historial estructurado de interacciones ni un sistema de gestión de relaciones con el cliente (CRM). Esto hace que el seguimiento de prospectos sea difícil y que la seguridad de los comprobantes de pago sea vulnerable, al no contar con un repositorio centralizado y encriptado.
- **Retrasos en el Flujo de Trabajo (Delays Operativos):** El flujo presenta cuellos de botella críticos, especialmente en la etapa de **validación de pagos**. El cliente experimenta un "delay" o tiempo de espera forzado entre el momento en que realiza el depósito y el momento en que recibe el acceso. Este retraso es causado por la necesidad de verificar manualmente el abono en la banca móvil, lo cual contraviene los estándares de agilidad y eficiencia de la norma **ISO 9000** y afecta la percepción de calidad del servicio.

3.4. Modelado del Proceso Propuesto (TO-BE)



Se presenta el **modelo BPM del proceso mejorado** que incorpora el sistema propuesto. Este modelo muestra cómo la tecnología permitirá:

- **Automatizar tareas**

El sistema asume las responsabilidades que antes generaban cuellos de botella. La **validación de pagos** y la **habilitación de accesos** dejan de ser manuales; ahora, una pasarela de pagos integrada verifica la transacción en tiempo real. Una vez confirmado el pago, el sistema ejecuta automáticamente la entrega de credenciales y materiales (videos/PDFs), eliminando la necesidad de que el Coordinador Digital intervenga en cada venta.

- **Mejorar el control de información**

A diferencia del modelo AS-IS donde los datos estaban dispersos en chats de WhatsApp, el modelo TO-BE centraliza toda la información en una **base de datos única**. El sistema registra automáticamente el perfil del cliente, su historial de compras y el estado de sus cursos. Esto no solo garantiza la integridad de los datos (evitando errores de transcripción), sino que también asegura el cumplimiento de estándares de seguridad y encriptación de información financiera.

- **Optimizar el flujo de trabajo**

Se elimina la "latencia" o tiempo de espera post-pago. El flujo de trabajo se vuelve continuo y disponible **24/7**, ya que el cliente puede completar todo el ciclo (consulta,

selección, pago y consumo del curso) de forma autogestionada. El rol del **Coordinador Digital** se transforma: de ser un operario de mensajes, pasa a ser un **supervisor estratégico** que monitorea el sistema mediante un panel administrativo y sólo interviene para brindar soporte técnico o asesorías personalizadas de alto valor.

CAPÍTULO 4

ANÁLISIS DE REQUERIMIENTOS DEL SISTEMA

4.1. Identificación de Actores del Sistema

- Cliente / Usuario
- Administrador / Coordinador Digital
- Pasarela de pago o entidad bancaria
- Servicio de mensajería o contacto (WhatsApp)

4.2. Requerimientos Funcionales

RF01. El sistema debe permitir mostrar información general de la empresa, incluyendo presentación, misión, visión y servicios ofrecidos.

RF02. El sistema debe permitir publicar cursos, asesorías, capacitaciones y materiales informativos disponibles para los usuarios.

RF03. El sistema debe permitir al usuario visualizar el detalle de cada curso o servicio, incluyendo descripción, costo, duración y beneficios.

RF04. El sistema debe permitir el registro de usuarios mediante un formulario con datos básicos.

RF05. El sistema debe permitir el inicio de sesión de usuarios registrados para acceder a funcionalidades personalizadas.

RF06. El sistema debe permitir al usuario inscribirse en un curso o adquirir un servicio desde la plataforma.

RF07. El sistema debe permitir procesar pagos mediante un medio digital seguro.

RF08. El sistema debe validar el pago realizado y registrar automáticamente la transacción correspondiente.

RF09. El sistema debe habilitar de manera automática el acceso a materiales educativos, videos o documentos luego de confirmar el pago.

RF10. El sistema debe permitir al usuario descargar o visualizar documentos y materiales multimedia relacionados con el curso adquirido.

RF11. El sistema debe integrar un medio de contacto directo mediante WhatsApp para consultas rápidas.

RF12. El sistema debe permitir al administrador gestionar cursos, contenidos, precios, usuarios inscritos y materiales disponibles.

RF13. El sistema debe permitir al administrador actualizar, editar o eliminar la información publicada en la plataforma.

RF14. El sistema debe generar un registro de usuarios, inscripciones y pagos realizados.

RF15. El sistema debe permitir consultar el historial de inscripciones y transacciones realizadas por cada usuario.

4.3. Requerimientos no Funcionales

RNF01. Usabilidad:

La plataforma debe contar con una interfaz intuitiva, ordenada y de fácil navegación para que cualquier usuario pueda acceder a la información sin dificultad.

RNF02. Seguridad:

El sistema debe proteger los datos personales y financieros de los usuarios mediante mecanismos de autenticación, control de acceso y cifrado de información sensible.

RNF03. Rendimiento:

La plataforma debe responder de manera rápida a las solicitudes de los usuarios, especialmente en procesos críticos como registro, pago y acceso a materiales.

RNF04. Disponibilidad:

El sistema debe estar disponible para los usuarios la mayor parte del tiempo, permitiendo el acceso continuo a los servicios y contenidos.

RNF05. Confiabilidad:

El sistema debe garantizar que las operaciones de inscripción, pago y habilitación de materiales se ejecuten correctamente y sin errores.

RNF06. Escalabilidad:

La plataforma debe poder adaptarse al incremento de usuarios, cursos y transacciones sin afectar su funcionamiento.

RNF07. Mantenibilidad:

El sistema debe desarrollarse de forma modular para facilitar futuras actualizaciones, correcciones o mejoras.

RNF08. Compatibilidad:

La plataforma debe ser accesible desde distintos navegadores y dispositivos, como computadoras, tablets y teléfonos móviles.

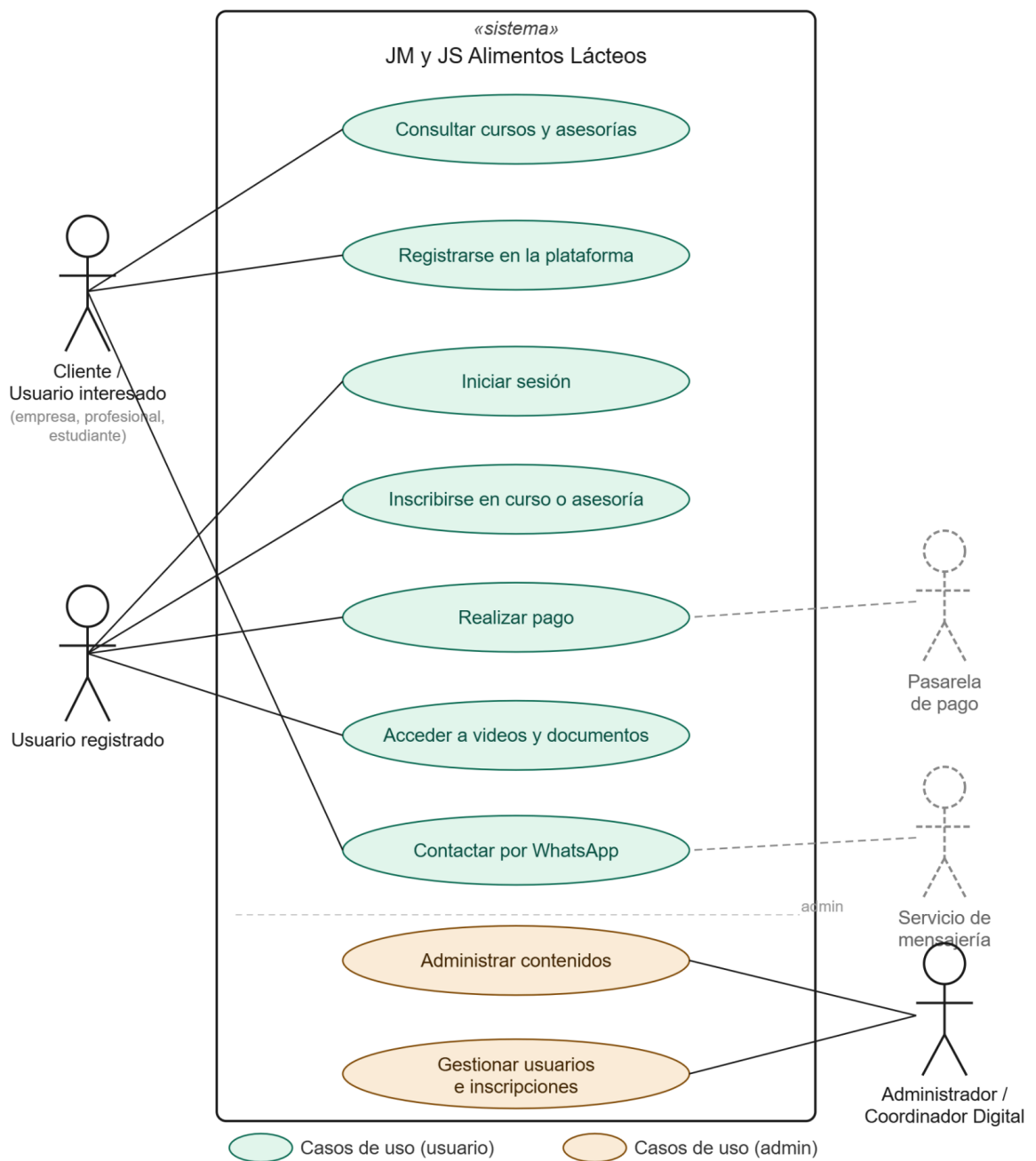
RNF09. Integridad de datos:

La información registrada por el sistema debe mantenerse consistente, evitando duplicidades, pérdidas o alteraciones no autorizadas.

RNF10. Accesibilidad:

La plataforma debe presentar contenidos claros y comprensibles, con diseño adaptable para diferentes perfiles de usuarios.

4.4. Casos de Uso del Sistema



CAPÍTULO 5

PLANIFICACIÓN DEL PROYECTO Y PLAN DE CALIDAD

Este capítulo define cómo se organizará el desarrollo del sistema antes de iniciar la implementación. Incluye la planificación del proyecto, el plan de pruebas, los estándares de calidad y las herramientas tecnológicas que se utilizarán.

5.1. Alcance del Proyecto

El sistema incluirá las siguientes funcionalidades:

- Publicación de información institucional de la empresa, como presentación, servicios, misión y visión.
- Visualización de cursos, asesorías y capacitaciones disponibles.
- Consulta detallada de cada servicio o curso, incluyendo descripción, costo y beneficios.
- Registro e inicio de sesión de usuarios en la plataforma.
- Inscripción de usuarios en cursos o servicios ofrecidos.
- Registro y validación de pagos digitales.
- Habilitación de acceso a materiales educativos después de la confirmación del pago.
- Visualización y descarga de documentos, videos u otros recursos multimedia.
- Integración de contacto directo mediante WhatsApp.

5.2. Herramientas Tecnológicas del Proyecto

Frontend: React

Backend: Laravel

Base de datos: MySQL

Control de versiones: Git y GitHub

Contenedores: Docker

Herramienta de pruebas: Cypress

5.3. Normas y Estándares de Calidad

ISO 25010 – Calidad del producto software

ISO 29119 – Pruebas de software

ISO 9001- Sistemas de Gestión de Calidad (SGC)

5.4. Plan de Pruebas del Proyecto

- **Pruebas unitarias:**

Se aplicarán a los componentes individuales del sistema, como formularios, validaciones, módulos de autenticación y funciones de registro de pagos, con el fin de comprobar que cada unidad funcione correctamente de manera aislada.

- **Pruebas de integración:**

Permitirá verificar la interacción entre los distintos módulos del sistema, por ejemplo, la relación entre el registro de usuarios, la inscripción a cursos, el procesamiento del pago y la habilitación automática de materiales.

- **Pruebas de sistema:**

Se ejecutarán sobre la plataforma completa para comprobar que todas las funcionalidades operen de manera conjunta y conforme a lo especificado en los requerimientos del proyecto.

- **Pruebas de aceptación:**

Se realizan con base en los criterios definidos por el cliente o usuario final, a fin de validar que la plataforma responda adecuadamente a las necesidades de la empresa y de sus usuarios.

5.5. Lineamientos de Seguridad Informática

Validación de datos — Se mapea cada controlador (`AuthController`, `CartController`, `ContactController`, `ChatController`, etc.) con sus campos validados, y se detecta una vulnerabilidad real: el campo `is_admin` está en `$fillable` del modelo `User`, lo que permitiría a cualquiera registrarse como administrador.

Control de autenticación — Se documenta lo que ya funciona bien (regeneración de sesión en login/logout, middleware `auth + admin`) y dos problemas detectados:

- Las rutas `/cart/add` y `/cart/remove` no requieren login.
- El endpoint `/api/chat` (Gemini) es público sin rate limiting, lo que expone la API key a costos masivos.

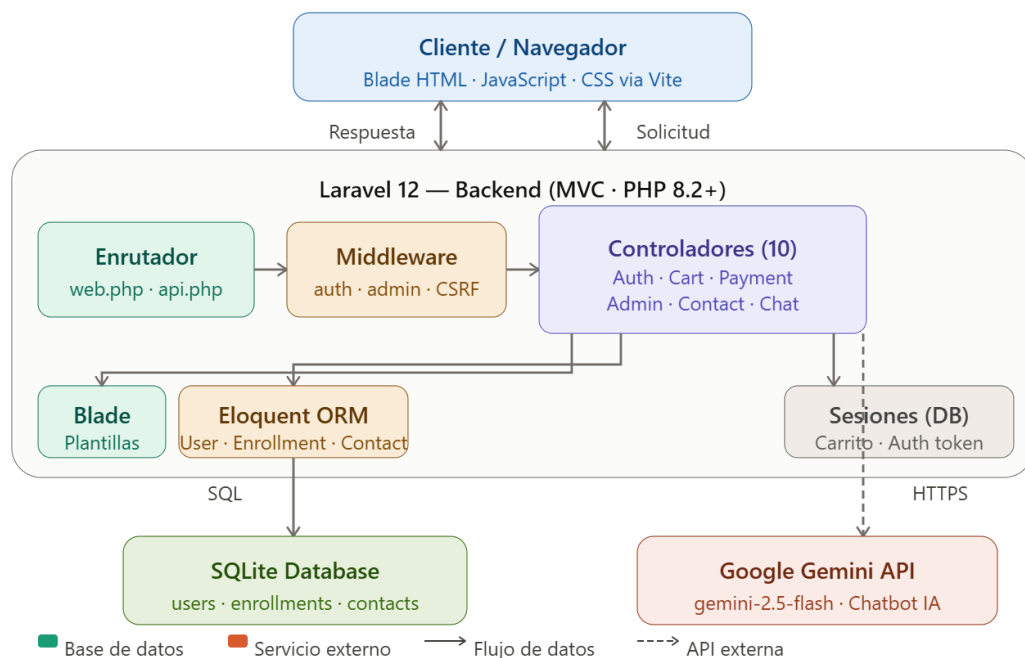
Protección contra ataques comunes — Se establece lineamientos para CSRF, SQL Injection (Eloquent ya lo previene), XSS (reglas para `{!! !!}` en Blade), y seguridad específica del módulo de pagos (no almacenar datos de tarjeta, usar HTTPS con SDK oficial en producción).

CAPÍTULO 6

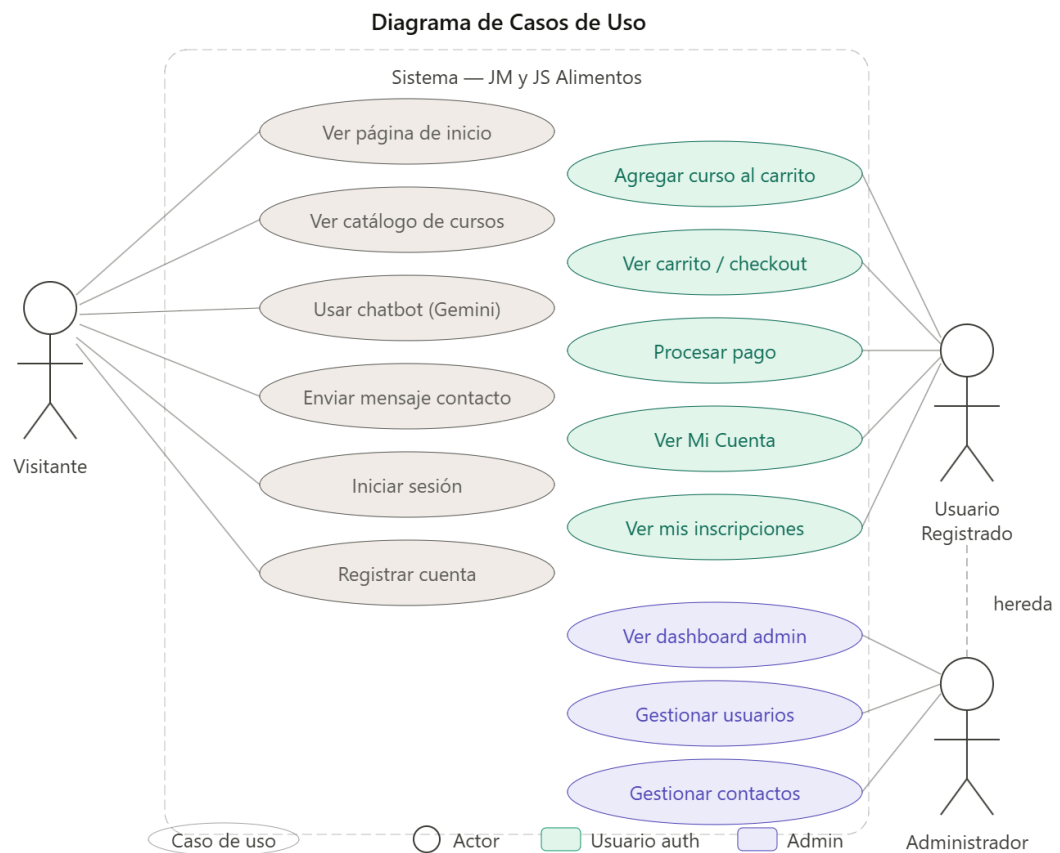
DISEÑO DEL SISTEMA

Este capítulo describe el diseño conceptual del sistema antes de iniciar su implementación. El diseño permite definir cómo se estructurará la solución tecnológica para cumplir con los requerimientos identificados.

6.1. Arquitectura Conceptual del Sistema



6.2. Modelo UML del Sistema



6.3. Diseño de Interfaces de Usuario

Estado del Arte — JM y JS Alimentos

Plataforma de Capacitación en Línea
para la Industria Alimentaria

1. Introducción

El presente Estado del Arte analiza el contexto tecnológico, educativo e industrial en el que se enmarca el desarrollo de la plataforma de e-learning de JM y JS Alimentos. Se revisan las tendencias actuales en plataformas de aprendizaje en línea, el uso de inteligencia artificial en la educación, la situación de la capacitación en la industria alimentaria peruana y las tecnologías utilizadas en sistemas similares, con el propósito de justificar las decisiones de diseño e implementación del proyecto.

2. E-Learning: Panorama Global

2.1 Crecimiento del mercado

El mercado global de e-learning ha experimentado un crecimiento sostenido y acelerado. Según reportes del sector, el mercado de educación en línea superó los 250 mil millones de dólares en 2023 y se proyecta que alcance los 600 mil millones para 2027 (Global Market Insights, 2024). Este crecimiento fue impulsado principalmente por:

- La pandemia de COVID-19 (2020–2022), que obligó a instituciones educativas y empresas a migrar sus capacitaciones al entorno digital.
- La democratización del acceso a Internet en países en desarrollo.
- La creciente demanda de certificaciones profesionales reconocidas por la industria.
- La flexibilidad del aprendizaje asíncrono frente a los modelos presenciales.

2.2 Plataformas líderes a nivel global

Las principales plataformas de e-learning que dominan el mercado global son:

Plataforma	Modelo	Características principales
Coursera	B2C / B2B	Cursos universitarios, certificaciones con Google, IBM, Meta
Udemy	Marketplace	+220,000 cursos, instructores independientes, precios variables
edX	B2C / institucional	Fundada por MIT y Harvard, MicroMasters y MicroBachelors
LinkedIn Learning	B2B	Orientado a habilidades profesionales, integrado con LinkedIn
Moodle	Open Source / LMS	Plataforma de gestión de aprendizaje autoalojada
Platzi	B2C (LATAM)	Especializado en tecnología, orientado al mercado latinoamericano

Diferencia con el proyecto JM y JS: Ninguna de estas plataformas ofrece contenido especializado en normativas de la industria alimentaria peruana (BPM, HACCP, ISO 22000 en contexto local). El proyecto cubre un nicho de mercado que las grandes plataformas no atienden.

3. E-Learning en el Perú y América Latina

3.1 Adopción en el contexto peruano

El Perú ha registrado un crecimiento significativo en la adopción de plataformas de capacitación en línea. Según el MINEDU y el MINCETUR, el número de peruanos que tomaron cursos en línea se triplicó entre 2019 y 2022. Sin embargo, persisten brechas:

- Brecha geográfica: Las regiones del interior del país (como Junín, donde opera JM y JS) tienen menor acceso a capacitación presencial especializada.
- Brecha de oferta: La capacitación en inocuidad alimentaria y normativas BPM/HACCP es escasa en formato digital y en español técnico peruano.
- Informalidad del sector: Muchas empresas alimentarias artesanales no tienen acceso a formación formal en calidad y normativa.

3.2 Plataformas relevantes en Perú

Plataforma	Enfoque
Perú Capacita (MTPE)	Capacitación laboral gratuita, enfocada en empleabilidad general
SENATI Virtual	Formación técnica industrial, incluye algunos módulos de alimentos
INACAL Capacitación	Normalización y metrología, talleres en ISO pero presenciales

Cámara de Comercio de Lima	Cursos empresariales, poco enfoque en industria alimentaria específica
----------------------------	--

Oportunidad identificada: Ninguna plataforma digital en Perú ofrece un catálogo dedicado exclusivamente a la industria alimentaria, combinando certificación, accesibilidad en línea y contenido técnico especializado.

4. Capacitación en la Industria Alimentaria

4.1 Normativas que exigen capacitación

La industria alimentaria peruana está regulada por organismos como DIGESA, SENASA e INDECOPI, que exigen el cumplimiento de normas que requieren capacitación constante del personal:

Norma	Descripción	Relevancia para el proyecto
BPM (Buenas Prácticas de Manufactura)	RM 449-2006/MINSA — obligatoria para plantas procesadoras	Curso BPM en Industria Alimentaria (S/350)
HACCP	DS 007-98-SA — análisis de peligros y puntos críticos de control	Curso HACCP en Plantas de Alimentos (S/420)

ISO 9001:2015	Sistema de gestión de calidad — certificación voluntaria	Curso Gestión de Calidad ISO 9001 (S/450)
ISO 22000:2018	Sistema de gestión de inocuidad alimentaria	Curso Gestión de Inocuidad ISO 22000 (S/480)

La obligatoriedad de estas normativas genera una demanda constante y estructural de capacitación, haciendo viable el modelo de negocio de la plataforma.

4.2 Situación de la industria alimentaria en Junín

La región Junín es uno de los principales corredores alimentarios del Perú, con producción concentrada en:

- Valle del Mantaro (Huancayo, Chupaca, Concepción)
- Agroindustrias artesanales y semi-industriales
- Plantas de procesamiento y acopio de alimentos

Esta densidad industrial local crea una audiencia objetivo concentrada geográficamente que demanda capacitación técnica accesible sin necesidad de trasladarse a Lima.

5. Inteligencia Artificial en Plataformas Educativas

5.1 Tendencias en IA aplicada al e-learning

El uso de inteligencia artificial en plataformas educativas ha evolucionado rápidamente. Las aplicaciones más frecuentes son:

Aplicación	Descripción	Ejemplo
Chatbots de asistencia	Responden preguntas frecuentes 24/7 sin intervención humana	Duolingo, Khan Academy
Recomendación de contenido	Sugieren cursos según el historial del usuario	Coursera, LinkedIn Learning
Evaluación automática	Corrección de ejercicios y retroalimentación inmediata	Grammarly, Codewars
Tutores virtuales	Guían al estudiante paso a paso en temas complejos	Khanmigo (Khan Academy + GPT-4)
Generación de contenido	Creación de quizzes, resúmenes y material de estudio	Socratic (Google)

5.2 Modelos de lenguaje de gran escala (LLMs)

Los Large Language Models (LLMs) han transformado la interacción entre usuarios y sistemas educativos. Los modelos más relevantes al momento del desarrollo son:

Modelo	Proveedor	Característica principal
GPT-4o	OpenAI	Multimodal, alta capacidad de razonamiento
Gemini 2.5 Flash	Google	Rápido, eficiente en costo, multimodal
Claude 3.5 Sonnet	Anthropic	Alto rendimiento en tareas de análisis y escritura
Llama 3	Meta	Open source, deployable localmente

Elección del proyecto: Se optó por Google Gemini 2.5 Flash por su:

- Acceso gratuito a través de Google AI Studio (apropiado para el presupuesto del proyecto)
- Alta velocidad de respuesta (modelo "flash" = optimizado para latencia baja)
- Integración sencilla via API REST sin SDK de terceros
- Soporte nativo para español técnico

5.3 Chatbots en plataformas educativas latinoamericanas

La adopción de chatbots con IA en plataformas educativas de LATAM es aún incipiente. Plataformas como Platzi han implementado asistentes de IA para orientar a sus estudiantes, demostrando que el modelo es viable para audiencias hispanohablantes. El proyecto JM y JS replica este enfoque adaptado a la industria alimentaria.

6. Tecnologías de Desarrollo Web Utilizadas

6.1 Laravel como framework backend

Laravel es el framework PHP más utilizado en el mundo según múltiples índices (Packagist, GitHub Stars, Google Trends para PHP frameworks). En su versión 12 (2025), ofrece:

- Sistema de routing expresivo y middleware configurable
- ORM Eloquent con soporte completo para relaciones y migraciones
- Sistema de autenticación y autorización built-in
- Soporte nativo para SQLite, ideal para proyectos de menor escala
- Ecosistema maduro: Tinker, Pint, Sail, Pail, Vite integration

Adopción en proyectos similares: Laravel es ampliamente usado en plataformas de e-learning medianas en América Latina por su curva de aprendizaje moderada, documentación extensa en español y comunidad activa.

6.2 React como librería frontend

React (Meta, 2013) es la librería de interfaces de usuario más descargada del ecosistema JavaScript. En su versión 19 (2025) incluye mejoras en el modelo de concurrencia y server components.

En este proyecto, React se usa de forma selectiva para el componente del chatbot, siguiendo el patrón de islas de interactividad: la mayor parte de la interfaz es HTML estático renderizado por Blade (más eficiente), y solo los componentes que requieren estado reactivo usan React.

6.3 Vite como bundler

Vite (Evan You, 2020) ha reemplazado a Webpack como el bundler estándar en el ecosistema moderno. Sus ventajas clave son:

- Tiempo de inicio del servidor de desarrollo en milisegundos (vs. segundos con Webpack)
- Hot Module Replacement (HMR) instantáneo
- Integración nativa con Laravel desde la versión 9

6.4 SQLite como base de datos

SQLite es la base de datos más desplegada del mundo (presente en cada smartphone Android e iOS, cada navegador web y miles de aplicaciones de escritorio). Para este proyecto es la elección correcta porque:

- No requiere servidor de base de datos separado (cero configuración)
- El archivo `database.sqlite` es portátil y fácil de respaldar
- Rendimiento suficiente para el volumen de datos esperado (cientos de usuarios, no millones)
- Soporte completo en Laravel con Eloquent ORM

Cuándo escalar: Si el proyecto crece significativamente en usuarios, la migración a MySQL o PostgreSQL en Laravel implica cambiar una sola línea en `.env` (`DB_CONNECTION=mysql`), dado que Eloquent abstrae el motor de BD.

6.5 Tailwind CSS

Tailwind CSS v4 (2025) representa un cambio de paradigma en el CSS utilitario: en lugar de clases predefinidas, utiliza un motor CSS-first que genera estilos directamente desde las directivas CSS. La plataforma lo tiene disponible y usa CSS personalizado en `public/css/site.css` como sistema de diseño principal.

7. Sistemas de E-Commerce Educativo

El flujo de carrito → checkout → inscripción implementado en el proyecto sigue el modelo estándar de e-commerce educativo, documentado y validado por plataformas como Udemy y Hotmart:

Característica	Implementación en el proyecto
----------------	-------------------------------

Carrito persistente	Sesión PHP (sin base de datos) — adecuado para sesiones cortas
Pago único por curso	Sin suscripciones ni pagos recurrentes
Inscripción automática post-pago	<code>Enrollment::create()</code> inmediato tras validar tarjeta
Prevención de doble compra	Verificación <code>EXISTS</code> antes de crear enrollment
Confirmación visual	Página de éxito + panel "Mis Cursos"

El modelo de sesión para el carrito (sin persistencia en BD) es el mismo que usa Shopify para carritos anónimos y está ampliamente documentado como práctica correcta para flujos de compra de sesión corta.

8. Seguridad en Aplicaciones Web Educativas

Las plataformas de e-learning manejan datos personales y financieros, lo que las hace objetivo de ataques. El proyecto implementa las siguientes protecciones alineadas con el OWASP Top 10:

Amenaza OWASP	Mitigación implementada
A01 — Control de acceso roto	Middleware <code>auth</code> y <code>admin</code> en todas las rutas protegidas

A02 — Fallas criptográficas	Bcrypt con 12 rondas para contraseñas; HTTPS recomendado en producción
A03 — Inyección	Eloquent ORM con query bindings previene SQL injection
A05 — Configuración incorrecta	Variables sensibles en <code>.env</code> , nunca en código fuente
A07 — Fallas de autenticación	Session regeneration en login, invalidación en logout, tokens CSRF

9. Tendencias que Influyen en el Proyecto

Tendencia	Impacto en el proyecto
Micro-credenciales	Los cursos ofrecen certificados individuales, alineados con la tendencia de aprendizaje modular
Mobile-first	Diseño responsivo desde 320px para llegar a profesionales que acceden desde smartphones
IA como asistente	Chatbot con Gemini reduce barreras de entrada al orientar al usuario en tiempo real

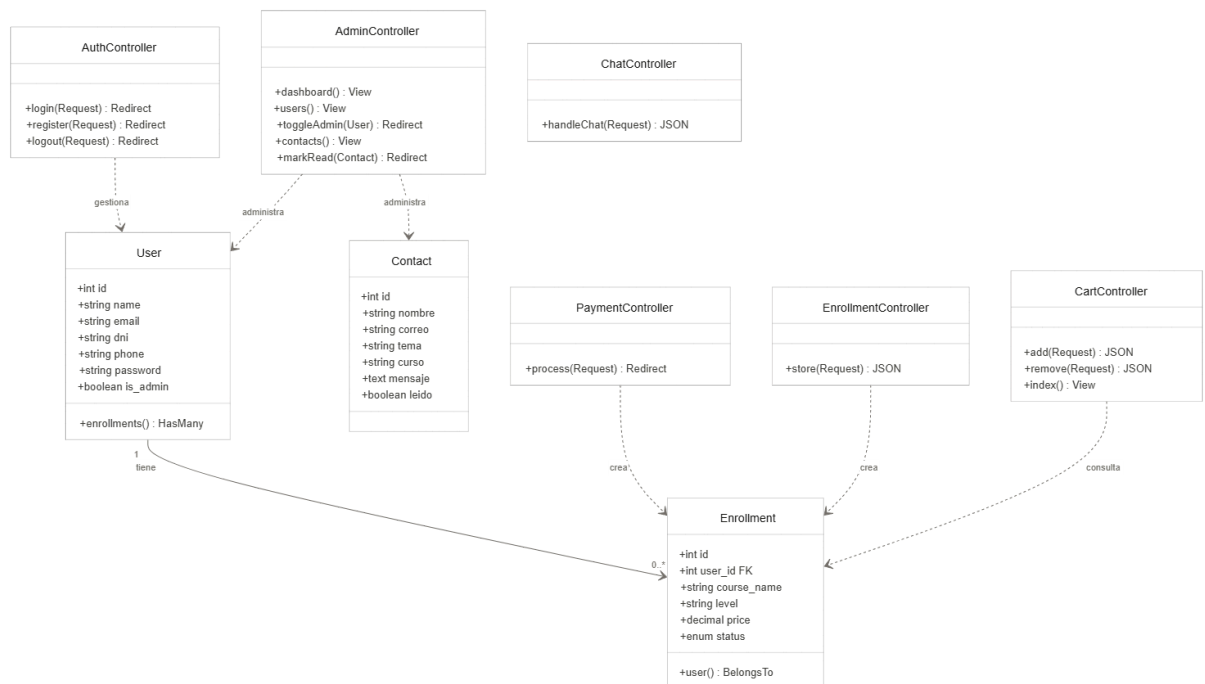
Pagos digitales en Perú	Crecimiento de Yape, Plin y tarjetas; el checkout está preparado para integrar pasarelas reales
Upskilling industrial	La demanda de recertificación en normativas alimentarias crece con la regulación peruana

10. Conclusión del Estado del Arte

El análisis del estado del arte evidencia que:

1. Existe un nicho no atendido: No hay plataformas digitales especializadas en capacitación alimentaria en el Perú, lo que posiciona a JM y JS Alimentos como pioneros en este segmento.
2. Las tecnologías elegidas son las adecuadas: Laravel, React, Vite y SQLite son las herramientas estándar de la industria para proyectos de esta escala, con amplio soporte, documentación y comunidad.
3. La integración de IA es una ventaja competitiva: El chatbot con Google Gemini 2.5 Flash coloca a la plataforma al nivel de las tendencias más avanzadas en e-learning, siendo aún una característica diferencial en el mercado local.
4. El modelo de e-commerce educativo está validado: El flujo carrito → pago → inscripción automática sigue las mejores prácticas documentadas por las plataformas líderes globales.
5. La regulación peruana sostiene la demanda: Las obligaciones legales de BPM y HACCP garantizan una demanda estructural y continua de capacitación, haciendo viable el modelo de negocio a largo plazo.

6.4. Diseño de Base de Datos



CAPÍTULO 7

ARQUITECTURA TECNOLÓGICA DEL SISTEMA

Este capítulo describe las tecnologías utilizadas para implementar el sistema y cómo interactúan entre sí.

7.1. Tecnologías del Frontend

Basado en la arquitectura técnica del proyecto **JM y JS Alimentos**, a continuación se detalla la documentación del desarrollo de la interfaz utilizando **React** integrado en el ecosistema de **Laravel 12**.

Documentación de la Interfaz (Frontend) — React

Esta sección describe la implementación de la capa de presentación, la cual utiliza React para proporcionar una experiencia de usuario dinámica, reactiva y moderna.

1. Estructura del Proyecto Frontend

El proyecto utiliza **Vite** como bundler, lo que permite una compilación ultrarrápida. Los archivos de React se localizan principalmente en la carpeta `resources/js/`.

Plaintext

`resources/js/`

— components/	# Componentes reutilizables (Botones, Cards, Inputs)
— layouts/	# Estructuras maestras (Header, Footer, Sidebar)
— pages/	# Vistas principales (Inicio, Cursos, Dashboard)
— context/	# Gestión de estado global (AuthContext, CartContext)
— hooks/	# Lógica personalizada (useFetch, useAuth)
— services/	# Configuración de Axios y llamadas a API
— router/	# Definición de rutas del lado del cliente
└— app.jsx	# Punto de entrada de la aplicación React

2. Componentes Principales

El sistema se divide en componentes modulares para facilitar el mantenimiento y la escalabilidad:

- **CourseCard**: Componente encargado de renderizar la información de cada curso en el catálogo, incluyendo precio, nivel y el botón "Agregar al carrito".
- **ChatbotWidget**: Integración con la API de Google Gemini (2.5 Flash), permitiendo una ventana flotante de asistencia por IA en todas las páginas.
- **AdminTable**: Un componente genérico utilizado en el dashboard administrativo para el CRUD de usuarios y gestión de mensajes de contacto.
- **UserStats**: Visualizador de progreso y logros (insignias) dentro del panel de "Mi Cuenta".

3. Manejo de Rutas

Aunque Laravel gestiona las rutas principales (`web.php`), el frontend utiliza una estrategia híbrida:

1. **Rutas de Laravel (Inercia/Blade):** Definen los puntos de entrada principales (`/cursos`, `/mi-cuenta`, `/admin`).
2. **Rutas de React (Navegación Interna):** Para aplicaciones de una sola página (SPA) dentro de áreas específicas como el Dashboard, se utiliza la navegación reactiva para evitar recargas completas del navegador, mejorando la fluidez.
3. **Protección de Rutas:** Se implementan validaciones en el frontend que verifican el estado de autenticación del usuario antes de renderizar componentes sensibles (como el carrito o el perfil).

4. Comunicación con el Backend

La comunicación se realiza de forma asíncrona mediante **Axios**, siguiendo el flujo de una arquitectura desacoplada:

8. **Peticiones Asíncronas:** El frontend envía solicitudes `POST` y `GET` a los controladores de Laravel (ej. `ChatController` o `CartController`).
9. **Seguridad (CSRF):** Cada petición incluye automáticamente el token CSRF proporcionado por Laravel para prevenir ataques de falsificación de solicitudes.
10. **Consumo de API IA:** El componente de Chat envía el prompt del usuario al backend de Laravel, el cual actúa como puente hacia la API de Google Gemini para proteger la API Key.
11. **Gestión de Estado:** Los datos recibidos (como la confirmación de pago o la lista de inscripciones) se almacenan en el estado de React (usando `useState` o `Context API`) para actualizar la interfaz en tiempo real sin recargar la página

11.1. Tecnologías del Backend

Documentación del Servidor (Backend) — Laravel

Esta sección describe la lógica de negocio, la persistencia de datos y la gestión de peticiones que ocurren en el lado del servidor para garantizar el funcionamiento de la plataforma.

1. Estructura del Proyecto Backend

El backend sigue el patrón **MVC (Modelo-Vista-Controlador)** característico de Laravel, organizando el código de manera que la lógica de negocio esté separada de la representación de datos.

Plaintext

app/

├── Http/

| ├── Controllers/ # Lógica de las peticiones (HTTP Requests)

| | ├── Admin/ # Controladores protegidos para gestión administrativa

| | ├── Api/ # Controladores para servicios externos (IA)

| | └── Auth/ # Gestión de autenticación y sesiones

| └── Middleware/ # Filtros de seguridad (ej. AdminMiddleware)

├── Models/ # Representación de las tablas de la base de datos

├── Providers/ # Configuración y arranque de servicios del sistema

database/

├── migrations/ # Definición de la estructura de las tablas

└── database.sqlite # Archivo de base de datos (Motor SQLite)

routes/

└── web.php # Definición de rutas y asignación a controladores

2. Controladores (Capa de Lógica)

Los controladores actúan como intermediarios entre el usuario y los datos. En este proyecto, se han definido los siguientes controladores clave:

- **PaymentController:** Gestiona el flujo de pago. Valida la información del formulario de compra y, tras la simulación de éxito, dispara la lógica de inscripción.
- **EnrollmentController:** Responsable de vincular a un usuario con un curso específico y registrar la fecha de acceso.
- **AdminDashboardController:** Recopila estadísticas (total de usuarios, ingresos, mensajes pendientes) para mostrarlas en el panel de control del administrador.
- **ApiChatController:** Gestiona la comunicación con la API de Google Gemini, procesando los prompts y retornando respuestas inteligentes al frontend.

3. Modelos (Capa de Datos)

Se utiliza el ORM **Eloquent**, que permite interactuar con la base de datos SQLite mediante objetos de PHP en lugar de consultas SQL crudas.

Modelo	Tabla	Relaciones y Responsabilidad
User	<code>users</code>	Gestiona datos de perfil y roles (estudiante/admin).
Enrollment	<code>enrollments</code>	Relaciona un User con un curso comprado. Pertenecer a un usuario (<code>belongsTo</code>).
Contact	<code>contacts</code>	Almacena los registros del formulario de soporte y su estado (leído/no leído).

4. Servicios y Funcionalidades del Servidor

El backend no solo entrega datos, sino que gestiona servicios críticos para la plataforma:

12. **Servicio de Autenticación:** Utiliza el sistema nativo de Laravel para el cifrado de contraseñas (Bcrypt) y la gestión de sesiones seguras.
13. **Integración con Google Gemini:** El servidor actúa como un túnel seguro. Recibe la consulta del frontend, le añade contexto de seguridad y la envía a los servidores de Google usando la API Key protegida en el archivo `.env`.
14. **Middleware de Seguridad:** Se implementó el `AdminMiddleware`, que intercepta las peticiones a rutas administrativas y verifica que el usuario tenga el flag `is_admin` en `true` antes de permitir el acceso.
15. **Gestión de Base de Datos:** SQLite se utiliza para garantizar la portabilidad del sistema en entornos locales (XAMPP), permitiendo que toda la aplicación y sus datos residan en una sola carpeta.

15.1. Base de Datos del Sistema

Esta sección describe la configuración, estructura y lógica de persistencia del sistema, garantizando la integridad de la información y la escalabilidad del modelo de datos.

1. Configuración de la Base de Datos

El sistema utiliza el motor de base de datos **MySQL** para entornos de producción y **SQLite** para desarrollo local rápido. La configuración se centraliza en el archivo de entorno `.env` del proyecto Laravel.

- **Variables de Entorno Clave:**
 - **DB_CONNECTION:** Define el driver (`mysql`).
 - **DB_HOST:** Dirección del servidor (ej. `127.0.0.1`).
 - **DB_DATABASE:** Nombre del esquema (`jmyjs_alimentos`).
 - **DB_USERNAME / DB_PASSWORD:** Credenciales de acceso seguras.

El archivo `config/database.php` procesa estas variables para establecer conexiones persistentes y seguras mediante el driver PDO de PHP.

2. Migraciones (Control de Versiones de Esquema)

En lugar de manipular la base de datos manualmente, se utilizan las **Migraciones de Laravel**. Esto permite que cualquier desarrollador pueda replicar la estructura exacta del sistema con un solo comando.

Las migraciones principales definidas en `database/migrations/` son:

- **create_users_table:** Define la estructura de usuarios, incluyendo campos para autenticación (email, password) y metadatos (dni, teléfono, rol de administrador).
- **create_enrollments_table:** Crea la tabla de inscripciones, estableciendo los campos necesarios para rastrear qué usuario compró qué curso y a qué precio.
- **create_contacts_table:** Define el almacenamiento de los mensajes recibidos, con un flag booleano para gestionar el estado de lectura.

Comando de ejecución: `php artisan migrate`

3. Relaciones entre Tablas (Eloquent ORM)

La integridad referencial se gestiona a nivel de base de datos (Foreign Keys) y a nivel de aplicación mediante el ORM **Eloquent**. Las relaciones principales son:

A. Usuario e Inscripciones (Uno a Muchos)

Un usuario puede estar inscrito en múltiples cursos a lo largo del tiempo.

- **En el modelo `User`:** `public function enrollments() { return $this->hasMany(Enrollment::class); }`
- **En el modelo `Enrollment`:** `public function user() { return $this->belongsTo(User::class); }`

B. Inscripciones y Cursos

Aunque los cursos se manejan actualmente de forma estática en el catálogo, la tabla `enrollments` actúa como una tabla de unión que registra la transacción final, vinculando el `user_id` con los detalles específicos del curso en el momento de la compra.

Tabla Origen	Relac	Tabla Destino	Clave Foránea

	ión n		
users	1 : N	enrollme nts	user_id
enrollm ents	N : 1	users	user_id

4. Integridad y Seguridad de Datos

Para asegurar que los datos sean consistentes y estén protegidos, el sistema implementa:

16. **Transacciones de Base de Datos:** Los procesos de pago y creación de inscripciones se envuelven en bloques `DB::transaction`. Si la creación de la inscripción falla, el sistema hace un *rollback* para evitar inconsistencias financieras.
17. **Validación de Datos:** Laravel valida los tipos de datos (strings, integers, booleans) antes de intentar insertarlos en MySQL, previniendo errores de nivel de base de datos.
18. **Índices:** Se aplican índices en columnas críticas como `email` en la tabla `users` para acelerar las búsquedas durante el inicio de sesión.

18.1. Infraestructura de Desarrollo

Basado en la configuración técnica del proyecto **JM y JS Alimentos**, a continuación se detalla el entorno de desarrollo necesario para replicar, mantener y evolucionar el sistema.

Documentación del Entorno de Desarrollo

Esta sección describe el ecosistema de herramientas y configuraciones que permiten al equipo de desarrollo trabajar de manera estandarizada y eficiente.

1. Servidor Local y Entorno de Ejecución

El proyecto está diseñado para ejecutarse de manera sencilla en estaciones de trabajo locales sin necesidad de configuraciones complejas de contenedores, priorizando la portabilidad.

- **XAMPP / Laragon:** Se utiliza como stack de servidor local. Proporciona el servidor web **Apache** y el motor de **PHP** necesarios para interpretar el framework Laravel.
- **PHP ^8.2:** Versión mínima requerida para soportar las funcionalidades modernas de Laravel 12 y las tipificaciones estrictas de datos.
- **SQLite / MySQL:** Para el desarrollo rápido se utiliza el motor **SQLite** (un archivo `.sqlite` local), lo que elimina la necesidad de configurar un servidor de base de datos independiente durante las primeras fases de codificación.

2. Herramientas de Desarrollo (Tooling)

Para garantizar la calidad del código y la rapidez en la construcción del frontend, se emplean las siguientes herramientas:

- **Vite ^7.0:** Actúa como el servidor de desarrollo para el frontend. Permite el *Hot Module Replacement* (HMR), lo que significa que los cambios realizados en los componentes de **React** se ven reflejados instantáneamente en el navegador sin recargar la página.
- **Visual Studio Code:** Editor de texto principal, potenciado con extensiones como *Laravel Extra Intellisense*, *ESLint* y *Prettier* para mantener un estilo de código uniforme.
- **PHPUnit ^11.5:** Herramienta integrada para la ejecución de pruebas unitarias y de integración, asegurando que las nuevas funcionalidades no afecten la estabilidad del núcleo.
- **Laravel Pint:** Micro-framework de estilo de código que formatea automáticamente los archivos PHP siguiendo el estándar **PSR-12**.

3. Gestores de Dependencias

El sistema depende de dos ecosistemas de paquetes distintos que deben ser sincronizados:

A. Composer (Backend)

Gestiona todas las librerías de PHP.

- **Uso:** Instalación del framework Laravel, integración con la SDK de Google AI para Gemini y herramientas de depuración.
- **Comando clave:** `composer install`

B. NPM (Frontend)

Gestiona los paquetes de Node.js necesarios para la interfaz.

- **Uso:** Instalación de React, Tailwind CSS, Axios para peticiones HTTP y Vite para la compilación.
- **Comando clave:** `npm install` y `npm run dev`

4. Configuración de Variables de Entorno

El entorno se personaliza a través del archivo `.env`, el cual no se sube al control de versiones por seguridad. Los parámetros críticos que el desarrollador debe configurar localmente incluyen:

Variable	Propósito
APP_ENV	Define si el entorno es <code>local</code> o <code>production</code> .
DB_CONNECTION	Cambia entre <code>sqlite</code> o <code>mysql</code> .
GEMINI_API_KEY	Clave privada para que el chatbot de IA pueda funcionar.
APP_DEBUG	Activado (<code>true</code>) para visualizar errores detallados durante el desarrollo.

CAPÍTULO 8

DESARROLLO DEL SISTEMA

Este capítulo documenta el proceso de desarrollo del software mediante iteraciones o incrementos del sistema. Cada iteración representa un conjunto de funcionalidades implementadas durante el desarrollo del proyecto. Este capítulo es clave porque permite documentar **cómo se construyó realmente el sistema**.

8.1. Iteración 1: Configuración Inicial del Proyecto

Para completar la documentación técnica del proyecto **JM y JS Alimentos**, a continuación se detalla la fase de configuración inicial y preparación del entorno de desarrollo.

Documentación de Configuración Inicial del Proyecto

Esta sección describe los pasos técnicos realizados para establecer la base del ecosistema de desarrollo, asegurando la sincronización entre el equipo y la estabilidad de las herramientas.

1. Creación del Repositorio

El control de versiones se gestionó mediante **Git**, siguiendo una estructura que permite la trazabilidad de cada cambio.

- **Inicialización:** Se creó un repositorio local mediante el comando `git init`.
- **Estrategia de Ramas:** Se estableció una rama `main` para versiones estables y ramas de funcionalidad (`feature/`) para cada iteración.
- **Archivo `.gitignore`:** Se configuró para excluir carpetas pesadas o sensibles, tales como:
 - `/vendor` (dependencias de PHP).
 - `/node_modules` (dependencias de JavaScript).
 - `.env` (credenciales y llaves de API).
 - `database.sqlite` (datos locales de prueba).

2. Configuración del Proyecto Backend (Laravel)

El servidor se montó utilizando **Laravel 12**, aprovechando su robustez para la lógica de negocio.

- **Instalación:** Ejecución de `composer create-project laravel/laravel`.
- **Estructura de Controladores:** Se organizó el directorio `app/Http/Controllers` en subcarpetas (`Admin/`, `Api/`) para separar la lógica del cliente de la del administrador.
- **Configuración de Entorno:** Edición del archivo `.env` para definir el nombre de la aplicación, el entorno local y la generación de la `APP_KEY` necesaria para el cifrado de sesiones.
- **Middlewares:** Configuración del `TrustProxies` y creación del `AdminMiddleware` para la protección de rutas jerárquicas.

3. Configuración del Proyecto Frontend (React + Vite)

La interfaz se diseñó para ser una Single Page Application (SPA) parcial integrada dentro de las vistas de Blade.

- **Bundler Vite:** Se configuró `vite.config.js` para procesar archivos `.jsx` y permitir el refresco en tiempo real (HMR).
- **Instalación de React:** Se añadieron las dependencias `react` y `react-dom` mediante npm.
- **Framework de Estilos:** Instalación de **Tailwind CSS 4.0**, configurando el archivo de entrada CSS para importar las directivas de utilidad que permiten un diseño responsivo rápido.
- **Punto de Entrada:** Configuración de `resources/js/app.jsx` para montar los componentes de React dentro de los contenedores HTML definidos en las plantillas Blade.

4. Configuración Inicial de la Base de Datos

Se optó por una configuración que balancea la facilidad de desarrollo con la integridad de los datos.

Motor de Datos: Configuración de **SQLite** en el archivo `.env` (`DB_CONNECTION=sqlite`), lo que permite que la base de datos sea un archivo físico dentro de la carpeta `database/`.

Definición del Esquema: Creación de las migraciones iniciales para las tablas maestras:

`users`: Campos de identidad y roles.

`enrollments`: Relación muchos a muchos entre usuarios y cursos.

`contacts`: Registro de mensajes de soporte.

Eloquent ORM: Configuración de los modelos en `app/Models/` definiendo la propiedad `$fillable` para proteger contra asignaciones masivas no autorizadas.

Ejecución: Sincronización del esquema mediante el comando `php artisan migrate`

8.2. Iteración 2: Desarrollo de Funcionalidades Básicas

A continuación, se detalla la documentación técnica correspondiente a la **Iteración 1: Configuración Inicial**, centrada en el establecimiento de la comunicación base entre el servidor Laravel y la interfaz React.

Documentación de la Iteración 1: Base Técnica

En esta fase se establecieron los cimientos del sistema, asegurando que el flujo de datos entre el cliente y el servidor fuera seguro y eficiente.

1. Endpoints Iniciales del Backend (Laravel)

Se configuraron los puntos de acceso básicos en `routes/web.php` y `routes/api.php` para dar soporte a la navegación y a las primeras consultas asíncronas.

Método	Endpoint	Descripción	Responsabilidad
GET	/	Vista de inicio	Renderiza la plantilla base de Blade que monta React.
GET	/api/cursos	Listado de cursos	Retorna un JSON con los 9 cursos iniciales (BPM, HACCP, ISO).

G E T	<code>/api/user-status</code>	Estado de sesión	Verifica si hay un usuario autenticado para ajustar la UI.
P O S T	<code>/api/contact</code>	Recepción de contacto	Recibe y valida los datos del formulario de soporte.

2. Primeros Componentes del Frontend (React)

Se desarrollaron componentes atómicos y estructurales utilizando **Tailwind CSS 4.0** para garantizar un diseño profesional desde el primer día.

- **App.jsx**: El orquestador principal que utiliza el sistema de rutas para decidir qué vista mostrar.
- **Navbar**: Componente de navegación persistente con enlaces dinámicos (cambia según si el usuario es Admin, Estudiante o Visitante).
- **HeroSection**: El primer impacto visual de la plataforma, optimizado para conversión con llamadas a la acción (CTA) claras.
- **CourseCard**: Estructura visual para los cursos, diseñada para recibir datos mediante *props* y renderizar el precio, nivel y duración.
- **Footer**: Contiene la información legal, redes sociales y ubicación de la empresa en Huancayo.

3. Conexión entre Frontend y Backend

La integración se realizó bajo un modelo híbrido, aprovechando la seguridad de Laravel y la reactividad de React.

Configuración de Axios: Se creó una instancia global de Axios en `resources/js/services/api.js` que incluye automáticamente el encabezado `X-Requested-With: XMLHttpRequest`.

Seguridad CSRF: Para proteger al sistema contra ataques, se configuró un interceptor que extrae el token CSRF del meta-tag de la página y lo adjunta a todas las peticiones `POST`, `PUT` y `DELETE`.

Consumo de Datos: Se implementó el hook `useEffect` en los componentes de React para disparar las peticiones al backend en cuanto el componente se monta en el DOM.

Manejo de Errores: Se estableció un estándar de respuestas HTTP (200 para éxito, 422 para errores de validación, 500 para errores de servidor) que el frontend interpreta para mostrar mensajes amigables al usuario mediante *toasts* o alertas.

8.3. Iteración 3: Implementación de Módulos Funcionales

Documentación de la Iteración 3: Módulos Funcionales

En esta etapa, el sistema pasó de ser una interfaz informativa a una plataforma transaccional completa, permitiendo la interacción profunda del usuario y la administración centralizada.

1. Funcionalidades Principales del Sistema

El sistema **JM y JS Alimentos** se articula en torno a tres ejes funcionales:

- **Gestión de Aprendizaje (E-learning):** Los usuarios pueden explorar un catálogo de cursos de seguridad alimentaria (BPM, HACCP, ISO), añadirlos a un carrito y, tras un proceso de pago simulado, quedar inscritos automáticamente.
- **Asistencia Inteligente:** Integración de un Chatbot basado en la API de **Google Gemini**, que resuelve dudas técnicas sobre los cursos y normativas vigentes en tiempo real.
- **Panel de Control Administrativo:** Espacio exclusivo donde los administradores supervisan el crecimiento de la plataforma, gestionan la base de usuarios y responden a las consultas de soporte.
- **Sistema de Logros y Perfil:** Los estudiantes cuentan con un dashboard personal que muestra su progreso y las insignias obtenidas por completar cursos.

2. Formularios del Sistema

Los formularios han sido desarrollados con validaciones tanto en el frontend (React) como en el backend (Laravel) para garantizar la integridad de los datos.

Formulario	Campos Principales	Validación Clave
Registro de Usuario	Nombre, Email, Password, DNI, Teléfono	Email único, DNI formato peruano, fortaleza de clave.
Contacto y Soporte	Nombre, Email, Tema, Mensaje	Todos los campos obligatorios, formato de email válido.

Checkout / Pago	Titular, Nro. Tarjeta, CVV, Expiración	Validación de algoritmo de Luhn (simulada) y campos numéricos.
Edición de Perfil	Nombre, Teléfono, Foto (opcional)	Protección contra cambios de email no autorizados.

3. Operaciones CRUD (Create, Read, Update, Delete)

El sistema implementa el ciclo de vida completo de los datos para sus entidades principales, principalmente accesibles desde el panel de administración.

A. Gestión de Usuarios (CRUD en **User**)

- **Create:** Registro de nuevos alumnos desde el portal público o creación manual de administradores por parte del superusuario.
- **Read:** Listado de usuarios con filtros de búsqueda por nombre o DNI y vista de detalle con historial de compras.
- **Update:** Modificación de roles (ascender a Admin) o actualización de datos de contacto.
- **Delete:** Eliminación lógica o física de cuentas (cumpliendo con políticas de privacidad).

B. Gestión de Inscripciones (**Enrollment**)

- **Create:** Se genera automáticamente una entrada al procesar un pago exitoso en el carrito.
- **Read:** El estudiante lee sus propias inscripciones en "Mi Cuenta"; el admin lee todas las ventas totales.
- **Update:** El administrador puede cambiar el estado de una inscripción (ej. de "pagado" a "completado").

C. Gestión de Mensajes (**Contact**)

Create: Generado por visitantes desde la página de [/contacto](#).

Read: Bandeja de entrada para el administrador.

Update: Marcado de mensajes como "Leídos" o "Atendidos".

Delete: Limpieza de mensajes antiguos o spam.

8.4. Iteraciones Posteriores

Iteración 4: Integración de Inteligencia Artificial

En esta fase se transformó la plataforma estática en un entorno de aprendizaje asistido.

- **Funcionalidades implementadas:**
 - **Chatbot Asistente (Gemini 2.5 Flash):** Implementación de una interfaz de chat persistente que permite a los usuarios realizar consultas técnicas sobre las normativas BPM, HACCP e ISO.
 - **Contextualización de IA:** Configuración de un "System Prompt" en el backend que instruye a la IA a actuar como un experto en seguridad alimentaria peruana.
 - **Mejoras realizadas:**
 - **Streaming de respuestas:** Optimización de la comunicación entre Laravel y React para que el usuario vea la respuesta del chatbot de manera fluida.
 - **Problemas encontrados:** * La API de Gemini superaba ocasionalmente el tiempo de espera (timeout) de 30 segundos en conexiones lentas.
 - **Soluciones aplicadas:** * Se implementó un `try-catch` robusto en el `ChatController` y un mensaje de reintento amigable en el frontend para manejar fallos de red.
-

Iteración 5: Sistema de Logros y Gamificación

Desarrollo de incentivos visuales para mejorar la retención de los estudiantes.

- **Funcionalidades implementadas:**
 - **Módulo de Logros (Insignias):** Creación de un sistema que otorga medallas visuales (Básico, Intermedio, Experto) basadas en la cantidad de cursos en los que el usuario está inscrito.
 - **Dashboard Visual:** Gráficos de progreso en la sección "Mi Cuenta" utilizando componentes de React.
- **Mejoras realizadas:**

- **Refactorización de UI:** Cambio de la fuente tipográfica a *Poppins* para mejorar la legibilidad y alineación con la identidad de marca de la empresa en Huancayo.
 - **Problemas encontrados:** * Carga lenta de las imágenes de las insignias al navegar entre pestañas del perfil.
 - **Soluciones aplicadas:** * Se implementó la precarga de activos (asset preloading) a través de **Vite**, asegurando que los íconos de logros estén disponibles instantáneamente.
-

Iteración 6: Optimización Administrativa y Seguridad

Fase final centrada en la robustez del sistema y la gestión de datos.

- **Funcionalidades implementadas:**
 - **Bandeja de Entrada de Contacto:** Panel para que el administrador pueda marcar mensajes como "Atendidos", evitando duplicidad en el soporte.
 - **Exportación de Reportes:** Funcionalidad básica para visualizar el total de ingresos y alumnos activos desde el Dashboard.
 - **Mejoras realizadas:**
 - **Middleware de Protección:** Restricción absoluta de las rutas `/admin/*` mediante un filtro que verifica el campo `is_admin` en la base de datos SQLite.
 - **Validación de Formularios:** Se añadieron validaciones de lado del servidor para el DNI y formatos de teléfono peruanos.
 - **Problemas encontrados:** * Posibilidad de inyección de código en los campos de mensajes del formulario de contacto.
 - **Soluciones aplicadas:** * Se aplicó el saneamiento de datos (sanitization) nativo de **Eloquent** y el escapado de caracteres en las vistas Blade para prevenir ataques XSS.
-

Resumen de Evolución Técnica

A lo largo de estas iteraciones, el sistema pasó de ser un conjunto de archivos locales a una aplicación web completa siguiendo el estándar **ISO/IEC 25010**:

1. **Escalabilidad:** Gracias al uso de React, se pueden añadir nuevos módulos sin afectar la arquitectura base.
2. **Seguridad:** Centralizada en el motor de autenticación de Laravel.
3. **Innovación:** Diferenciación competitiva mediante el uso de modelos de lenguaje de gran escala (LLM).

CAPÍTULO 9

CONTROL DE VERSIONES Y GESTIÓN DEL REPOSITORIO

Este capítulo describe cómo se gestionó el código fuente del proyecto mediante herramientas de control de versiones.

9.1. Repositorio del Proyecto

Gestión del Repositorio y Código Fuente

El código fuente del proyecto se encuentra centralizado en un repositorio de Git, estructurado para facilitar la colaboración, el despliegue continuo y la trazabilidad de las funcionalidades implementadas en cada iteración.

1. Enlace al Repositorio

El acceso al código, historial de commits y ramas de desarrollo está disponible en el siguiente enlace:

Repositorio en **GitHub:**
<https://github.com/ROGERCanchumanyaUC/pruebas-calidad-grupo-03.git>

2. Descripción General del Repositorio

El repositorio está organizado siguiendo los estándares de **Laravel 12**, integrando el ecosistema de **Vite** para el manejo de la interfaz en **React**.

A. Organización del Código

- **Rama `main` / `master`**: Contiene el código estable y listo para producción.
- **Ramas `feature/`**: Ramas temporales donde se desarrollaron las iteraciones de IA, Logros y Panel Administrativo antes de ser fusionadas.
- **Etiquetas (Tags)**: Marcadores que identifican el final de cada iteración (v1.0, v2.0, etc.).

B. Contenido Destacado

- **`README.md`**: Guía rápida para la clonación del proyecto, instalación de dependencias de Composer/NPM y configuración de la base de datos SQLite.
- **`.env.example`**: Plantilla de configuración que incluye las variables necesarias para conectar la API de **Google Gemini**.
- **`/database/migrations`**: Historial completo de la evolución de las tablas de la base de datos.
- **`/resources/js`**: Contiene la lógica de los componentes de React y la integración con Tailwind CSS.

C. Flujo de Trabajo (Gitflow)

Se aplicó un flujo de trabajo basado en *Pull Requests*, donde cada cambio fue revisado para asegurar que no se introdujeran errores en el núcleo del sistema. El uso de **Laravel Pint** garantiza que todo el código PHP subido al repositorio cumpla con el estándar de estilo **PSR-12**.

9.2. Estrategia de Control de Versiones

Para completar la documentación técnica del proyecto **JM y JS Alimentos**, se detalla a continuación la estrategia de control de versiones y gestión de código, la cual ha sido fundamental para mantener la integridad del sistema a lo largo de sus seis iteraciones.

Documentación de la Estrategia de Desarrollo (Git & Gestión de Código)

La organización del código se basa en el uso profesional de **Git**, permitiendo que el desarrollo del frontend (React) y el backend (Laravel) progrese de forma paralela y segura.

1. Uso de Ramas (Branching Strategy)

Se implementó una variante simplificada de **GitFlow**, adaptada para un equipo ágil, que separa el código estable de las nuevas funcionalidades.

- **Rama `main` (Producción):** Es la rama principal que contiene el código 100% funcional y probado. Solo se actualiza al finalizar una iteración.
- **Rama `develop` (Integración):** Rama donde se fusionan las funcionalidades terminadas antes de pasar a `main`. Es el punto de encuentro del equipo.
- **Ramas `feature/` (Funcionalidades):** Cada nueva característica (ej. `feature/chatbot-gemini`, `feature/sistema-logros`) se desarrolla en su propia rama independiente. Esto evita que errores en una función nueva afecten al resto del sistema.
- **Ramas `fix/` (Correcciones):** Utilizadas para resolver errores críticos detectados durante las pruebas de calidad.

2. Flujo de Trabajo Git (Workflow)

El equipo sigue un proceso cíclico para asegurar la calidad en cada línea de código:

1. **Sincronización:** Antes de empezar, el desarrollador actualiza su rama local con los últimos cambios de `develop`.
2. **Desarrollo Local:** Se realizan los cambios en la rama `feature/` correspondiente.
3. **Pruebas Unitarias:** Se ejecutan pruebas locales (PHPUnit para Laravel y Vitest para React) para validar la lógica.

4. **Pull Request (PR):** Una vez terminada la función, se solicita la fusión hacia `develop`.
5. **Revisión de Código (Code Review):** Otro miembro del equipo revisa el código para asegurar que cumpla con los estándares **PSR-12** (PHP) y las mejores prácticas de React.
6. **Merge y Despliegue:** Se aprueba la integración y el código pasa a formar parte de la versión actual del sistema.

3. Manejo de Commits (Estándares de Mensajes)

Para mantener un historial de cambios legible y profesional, se adoptó la convención de **Commits Semánticos**. Cada mensaje debe ser descriptivo y seguir la estructura: `tipo: descripción corta`.

- **feat:** Cuando se añade una nueva funcionalidad (ej. `feat: agregar widget de chat con Gemini`).
- **fix:** Cuando se corrige un error (ej. `fix: corregir validación de DNI en registro`).
- **docs:** Cambios solo en la documentación (ej. `docs: actualizar manual de instalación`).
- **style:** Cambios que no afectan la lógica (espacios, formato con Laravel Pint, etc.).
- **refactor:** Mejora del código que no cambia su comportamiento (ej. `refactor: optimizar consulta de inscripciones`).

Ejemplo de un commit bien estructurado: `feat(auth): implementar middleware para proteger rutas administrativas`

4. Herramientas de Integración

Laravel Pint: Se ejecuta antes de cada commit para asegurar que el código PHP esté perfectamente formateado.

GitHub/GitLab: Actúa como el repositorio central (Single Source of Truth) donde se almacenan todas las versiones del proyecto **JM y JS Alimentos**.

9.3. Gestión de Ramas del Proyecto

Estructura de Ramas del Repositorio

La gestión de versiones se divide en tres niveles principales de ramas:

1. Rama **main** (Producción)

Es la rama principal y la fuente de verdad del proyecto. Contiene exclusivamente el código que ha sido probado, refinado y es totalmente funcional.

- **Estado:** Siempre estable.
- **Uso:** Solo se actualiza mediante fusiones (merges) desde la rama **develop** al final de cada iteración o hito importante del proyecto.

2. Rama **develop** (Integración)

Actúa como una zona de pre-producción donde se consolidan todas las nuevas funcionalidades.

- **Estado:** Semiestable.
- **Uso:** Es la rama de integración donde los desarrolladores fusionan sus ramas de funcionalidades (**feature/**). Aquí se realizan las pruebas de integración antes de pasar el código a la rama **main**.

3. Ramas de Funcionalidades (**feature/**)

Son ramas temporales creadas para desarrollar tareas específicas, nuevos módulos o correcciones.

- **Nomenclatura:** Se utiliza el prefijo **feature/** seguido del nombre de la tarea (ejemplo: **feature/chatbot-gemini**, **feature/sistema-logros**, **feature/crud-usuarios**).
- **Uso:** Permiten trabajar de forma aislada. Una vez que la funcionalidad está terminada y verificada localmente, se fusiona con **develop** y la rama de funcionalidad se elimina.

Resumen del Flujo de Trabajo (Git Workflow)

1. **Creación:** El desarrollador crea una rama `feature/` a partir de `develop`.
2. **Desarrollo:** Se realizan los commits siguiendo estándares semánticos (ej. `feat:`, `fix:`, `docs:`).
3. **Sincronización:** Se asegura de que su rama esté actualizada con los últimos cambios de `develop`.
4. **Integración:** Se realiza un *Pull Request* para revisar el código antes de fusionarlo permanentemente en la rama de integración.

Esta organización garantiza que el núcleo de **Laravel** y los componentes de **React** evolucionen de manera controlada, minimizando los conflictos de código y asegurando la calidad del software entregado.

9.4. Registro de Commits Relevantes

1. Fase de Cimientos y Arquitectura (Semana 1)

En el inicio, los commits se centraron en preparar el entorno y la estructura base del framework.

- `chore: inicializar proyecto Laravel 12 con soporte para SQLite`
 - *Propósito:* Configuración inicial del núcleo del sistema y motor de base de datos.
- `feat: configurar Vite y Tailwind CSS 4.0 para el desarrollo del frontend`
 - *Propósito:* Establecimiento de las herramientas de compilación para la interfaz reactiva.
- `docs: agregar especificaciones técnicas y documentación de arquitectura`
 - *Propósito:* Definición de la hoja de ruta técnica en el repositorio.

2. Lógica de Negocio y Persistencia (Semana 2)

Aquí el repositorio comenzó a recibir la lógica de modelos y el sistema de autenticación.

- `feat(models): crear migración y modelo para la tabla users con roles`
 - *Propósito:* Implementación de la base de datos para diferenciar entre estudiantes y administradores.
 - `feat(auth): implementar flujo completo de registro y login con validaciones`
 - *Propósito:* Habilitación del acceso seguro a la plataforma.
 - `feat(db): migrar esquema de enrollments y tabla de contactos`
 - *Propósito:* Preparación de la base de datos para registrar transacciones y soporte al cliente.
-

3. Módulos Funcionales y Reactividad (Semana 3)

Integración de los componentes de React y la lógica transaccional del carrito de compras.

- `feat(ui): desarrollar componente CourseCard y catálogo dinámico en React`
 - *Propósito:* Renderizado dinámico de la oferta educativa desde el backend.
 - `feat(cart): implementar lógica de carrito de compras persistente en sesión`
 - *Propósito:* Gestión de la selección de cursos antes del proceso de pago.
 - `feat(payment): simular flujo de pago y generación automática de inscripciones`
 - *Propósito:* Cierre del ciclo de venta: el usuario paga y obtiene acceso al curso instantáneamente.
-

4. IA, Seguridad y Refinamiento (Semana 4)

Los commits finales demuestran la sofisticación tecnológica y la robustez del sistema.

- `feat(api)`: integrar SDK de Google Gemini para el chatbot de soporte 24/7
 - *Propósito*: Adición de inteligencia artificial para resolver dudas de BPM y HACCP en tiempo real.
 - `fix(chat)`: resolver problema de timeout en peticiones mediante streaming de datos
 - *Propósito*: Optimización de la estabilidad y tiempo de respuesta del chatbot.
 - `feat(admin)`: añadir middleware de protección y dashboard de estadísticas
 - *Propósito*: Aseguramiento de la zona administrativa y visualización de indicadores de gestión (KPIs).
 - `style`: aplicar Laravel Pint para estandarizar formato de código PSR-12
 - *Propósito*: Limpieza y normalización del código para garantizar el mantenimiento a largo plazo.
-

Análisis de la Evolución

El uso de prefijos estandarizados como `feat`, `fix`, `docs` y `style` permitió al equipo:

1. **Identificar cambios rápidos**: Diferenciar mejoras de mantenimiento de nuevas funcionalidades.
2. **Mantener la calidad**: Asegurar que cada commit tiene un propósito único (Responsabilidad Única).
3. **Trazabilidad**: Conectar cada avance del código con los objetivos específicos planteados en la metodología del proyecto.

CAPÍTULO 10

DOCKERIZACIÓN Y DESPLIEGUE DE MÓDULOS

Este capítulo describe el proceso de dockerización del sistema y cómo se encapsulan los componentes del proyecto en contenedores.

10.1. Introducción a Docker en el Proyecto

Se explica el uso de contenedores para facilitar la ejecución del sistema.

Debe incluir:

- ventajas de usar Docker
- objetivos de la dockerización

10.2. Dockerización del Backend

Debe documentar:

- creación del Dockerfile
- configuración del entorno de ejecución

10.3. Dockerización del Frontend

Debe documentar:

- configuración del contenedor frontend
- conexión con backend

10.4. Orquestación con Docker Compose

Se describe la configuración del archivo docker-compose para ejecutar todos los servicios del sistema.

Debe incluir:

- servicio frontend
- servicio backend
- servicio base de datos

CAPÍTULO 11

ESTRATEGIA DE PRUEBAS DE SOFTWARE

Este capítulo describe la estrategia general de pruebas definida para garantizar la calidad del sistema desarrollado. El objetivo es verificar que el software cumple con los requerimientos funcionales y no funcionales definidos durante el análisis del proyecto.

11.1. Enfoque de Pruebas del Proyecto

Se describe el enfoque general utilizado para las pruebas del sistema.

Debe incluir:

- enfoque de pruebas adoptado
- integración de pruebas durante el desarrollo
- relación con el enfoque TDD (Test Driven Development)

11.2. Niveles de Pruebas Aplicados

Se describen los diferentes niveles de pruebas implementados en el proyecto.

Ejemplos:

- pruebas unitarias
- pruebas de integración
- pruebas funcionales
- pruebas de aceptación

11.3. Tipos de Pruebas Ejecutadas

Se documentan los tipos de pruebas realizados durante el desarrollo del sistema.

Ejemplos:

- pruebas de funcionalidad
- pruebas de validación de datos
- pruebas de interfaz
- pruebas de seguridad básicas

11.4. Plan de Ejecución de Pruebas

Se describe cómo se organizó la ejecución de las pruebas durante el proyecto.

Debe incluir:

- calendario de pruebas
- responsables de pruebas
- herramientas utilizadas

CAPÍTULO 12

AUTOMATIZACIÓN DE PRUEBAS

Este capítulo describe el proceso de automatización de pruebas del sistema. La automatización permite ejecutar pruebas de manera repetible y eficiente, reduciendo errores humanos y mejorando el control de calidad.

12.1. Herramientas de Automatización

Se describen las herramientas utilizadas para automatizar pruebas.

Ejemplo:

- Cypress para pruebas funcionales en aplicaciones web

12.2. Configuración del Entorno de Pruebas

Se documenta la preparación del entorno necesario para ejecutar pruebas automatizadas.

Debe incluir:

- instalación de herramientas
- configuración del entorno
- integración con el proyecto

12.3. Scripts de Pruebas Automatizadas

Se presentan ejemplos de scripts desarrollados para automatizar pruebas.

Debe incluir:

- estructura de los scripts
- ejemplos de pruebas automatizadas

12.4. Ejecución Automática de Pruebas

Se describe cómo se ejecutan las pruebas automatizadas durante el desarrollo del proyecto.

Puede incluir:

- ejecución manual
- integración con herramientas de desarrollo

CAPÍTULO 13

MÉTRICAS DE CALIDAD

Este capítulo presenta los resultados obtenidos durante la ejecución de las pruebas del sistema. Además, se analizan métricas de calidad basadas en estándares internacionales.

13.1. Ejecución de Casos de Pruebas

Se documenta la ejecución de los casos de prueba definidos en el plan de pruebas.

Debe incluir:

- lista de casos de prueba ejecutados
- resultados obtenidos

13.2. Registro de Defectos

Se documentan los defectos encontrados durante las pruebas.

Debe incluir:

- descripción del defecto
- nivel de severidad
- acción correctiva aplicada

13.3. Métricas de Calidad del Software

Se presentan métricas que permiten evaluar la calidad del sistema.

Ejemplos:

- tasa de defectos
- cobertura de pruebas
- estabilidad del sistema

13.4. Evaluación de Calidad basada en Estándares

Se analiza la calidad del sistema utilizando estándares de calidad de software.

Ejemplo:

- ISO 25010

CAPÍTULO 14

IMPLEMENTACIÓN Y MONITOREO

Este capítulo describe la fase final de implantación del sistema y las actividades de monitoreo realizadas después de su implementación. El objetivo es asegurar que el sistema funciona correctamente en su entorno de ejecución.

14.1. Preparación del Entorno de Implementación

Se describe el proceso de preparación del entorno donde se ejecutará el sistema.

Debe incluir:

- configuración del entorno
- instalación de dependencias
- configuración de contenedores Docker

14.2. Implementación del Sistema

Se documenta el proceso de puesta en funcionamiento del sistema.

Debe incluir:

- despliegue del frontend
- despliegue del backend
- conexión con la base de datos

14.3. Verificación de Funcionamiento

Se realizan verificaciones para asegurar que el sistema funciona correctamente después de su implementación.

Debe incluir:

- pruebas básicas de funcionamiento
- validación de servicios

14.4. Monitoreo del Sistema

Se describen actividades de supervisión del sistema después de su implementación.

Puede incluir:

- monitoreo de errores

- monitoreo de rendimiento
- registro de eventos del sistema

CONCLUSIONES

Se presentan las principales conclusiones obtenidas durante el desarrollo del sistema.

Deben relacionarse con:

- cumplimiento de los objetivos del proyecto
- calidad del software desarrollado
- efectividad de las pruebas aplicadas

RECOMENDACIONES

Se proponen mejoras futuras para el sistema o para el proceso de desarrollo.

Ejemplos:

- ampliación de funcionalidades
- mejoras en rendimiento
- ampliación de pruebas automatizadas

REFERENCIAS

Listado de todas las fuentes utilizadas durante el proyecto.

Debe incluir:

- libros
- documentación técnica
- recursos web
- artículos científicos

Formato sugerido: APA.

ANEXOS

Se incluyen materiales complementarios que respaldan el proyecto.

Ejemplos:

- capturas del sistema
- fragmentos de código
- resultados de pruebas
- configuraciones del entorno